

DS - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/", RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4254 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 4162 values"
## [1] "92 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	536	35	123	NA	NA	694
2	65	NA	421	93	106	685
3	302	NA	159	206	15	682
4	299	NA	216	62	36	613
5	217	NA	201	65	37	520
6	195	1	129	69	20	414
7	172	NA	94	26	18	310
8	85	NA	53	22	4	164
9	71	NA	2	NA	7	80

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7723343	0.0504323	0.1772334	NA	NA	694
2	0.0948905	NA	0.6145985	0.1357664	0.1547445	685
3	0.4428152	NA	0.2331378	0.3020528	0.0219941	682
4	0.4877651	NA	0.3523654	0.1011419	0.0587276	613
5	0.4173077	NA	0.3865385	0.1250000	0.0711538	520
6	0.4710145	0.0024155	0.3115942	0.1666667	0.0483092	414
7	0.5548387	NA	0.3032258	0.0838710	0.0580645	310
8	0.5182927	NA	0.3231707	0.1341463	0.0243902	164
9	0.8875000	NA	0.0250000	NA	0.0875000	80

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position
```

```
syll_comp_plot <- ggplot(syll_comp_plot_data,
  aes(x=pos,y=percent,group=syll_component,
      color=syll_component,
      linetype = syll_component,
      shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=s
```

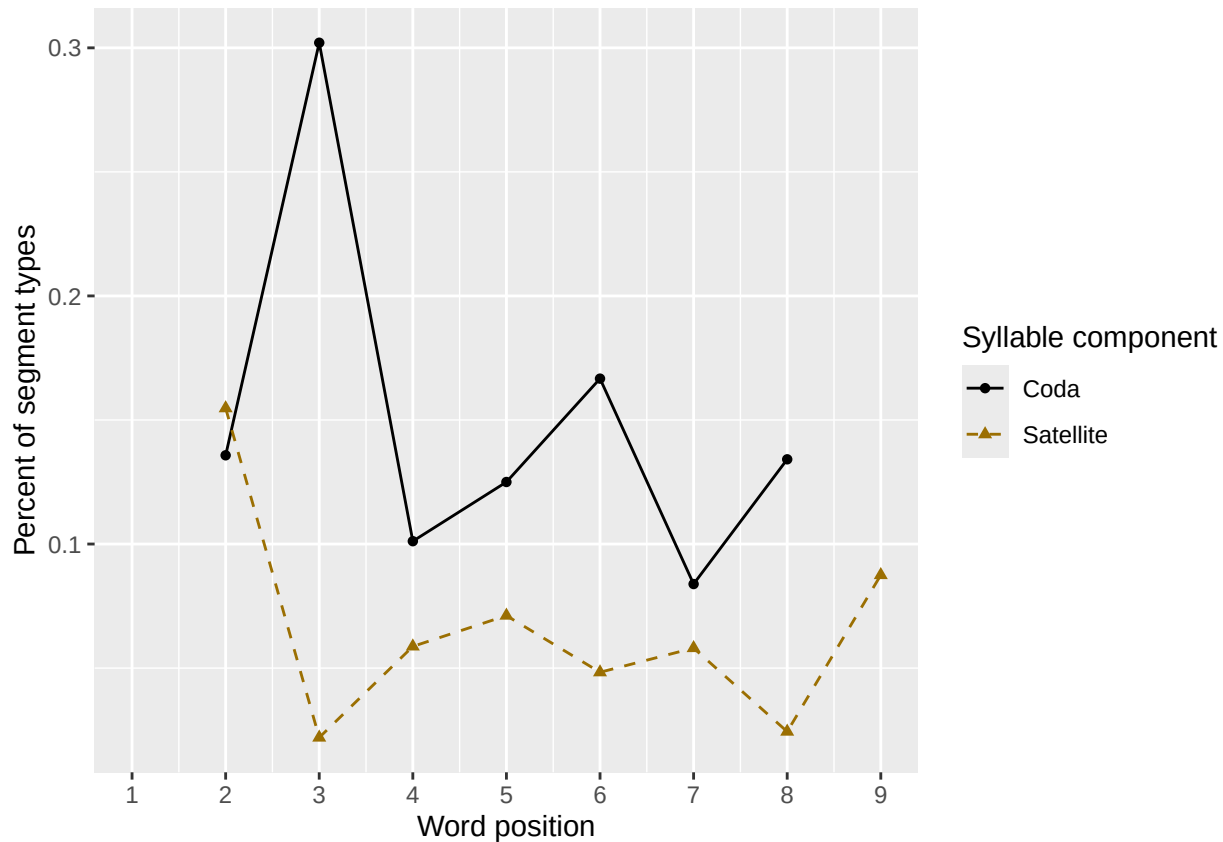
```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```



```
# len/pos table
```

```
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
```

```
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
```

```
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 1     1     1    NA    NA    NA    NA    NA    NA
## 2      5 1     0.946 0.925 0.925 NA    NA    NA    NA    NA
## 3      6 0.974 0.939 0.912 0.915 0.860 NA    NA    NA    NA
## 4      7 1     1     0.914 0.897 0.886 0.821 NA    NA    NA
## 5      8 0.986 0.959 0.938 0.854 0.828 0.845 0.793 NA    NA
## 6      9 0.965 0.928 0.928 0.915 0.855 0.8    0.795 0.774 NA
## 7     10 0.964 0.951 0.902 0.914 0.862 0.753 0.732 0.725 0.712

```

```
# len/pos table
```

```
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))

```

```
pos_len_N_table
```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    62    62    62    NA    NA    NA    NA    NA    NA
## 2      5    93    93    93    93    NA    NA    NA    NA    NA
## 3      6   116   114   113   106   107    NA    NA    NA    NA
## 4      7   108   106   105   107   105   106    NA    NA    NA
## 5      8   147   146   144   144   145   142   145    NA    NA
## 6      9    85    83    83    82    83    85    83    84    NA
## 7     10    83    81    82    81    80    81    82    80    80

```

```

obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")

```

```
pred_linetypes <- "dashed"
```

```
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
```

```
pos_len_summary$pos <- factor(pos_len_summary$pos)
```

```
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
```

```

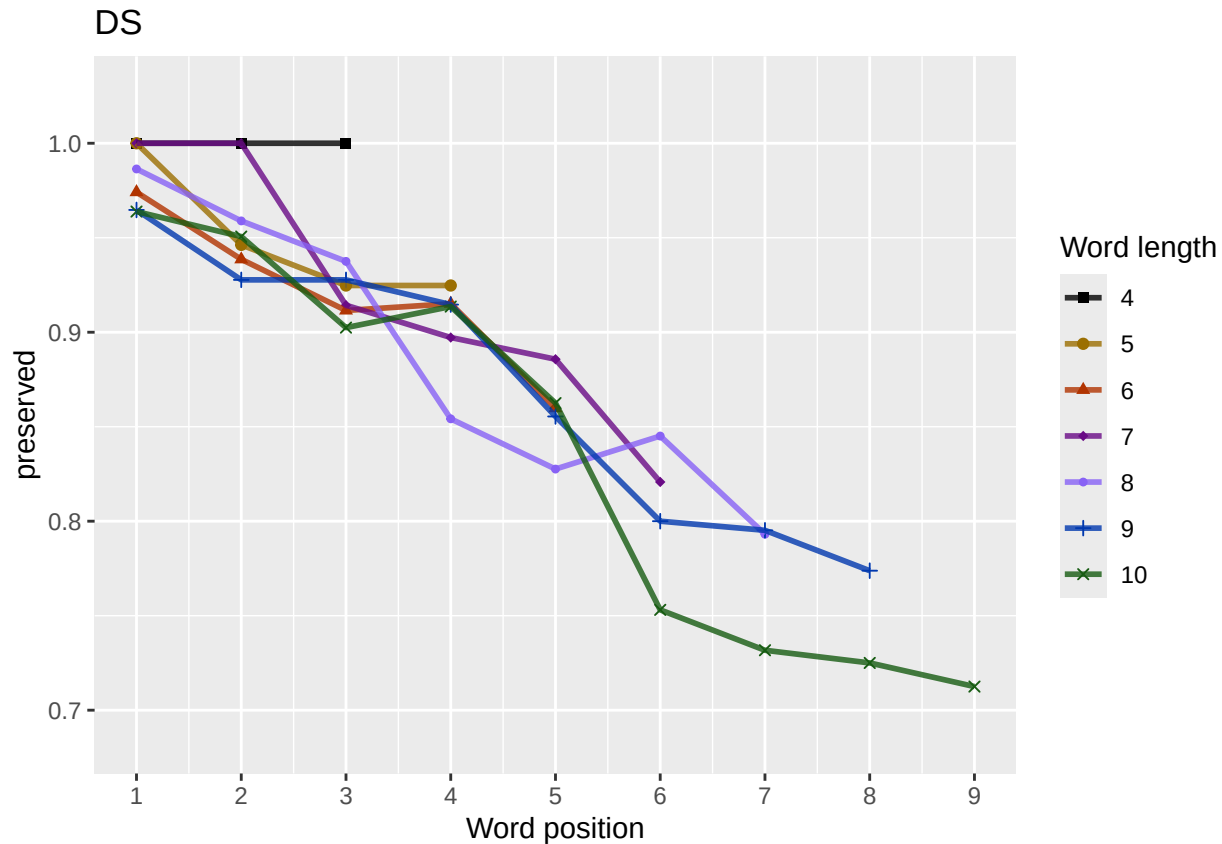
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
                      "preserved ~ stimlen",
                      "preserved ~ pos",
                      "preserved ~ stimlen + pos",
                      "preserved ~ stimlen*pos",
                      "preserved ~ I(pos^2)+pos",
                      "preserved ~ stimlen + I(pos^2) + pos",
                      "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 7
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos
##      5.38270      -0.10434      0.04774      -0.79716
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4158 Residual
## Null Deviance:      2792
## Residual Deviance: 2534 AIC: 2542
## log likelihood: -1267.191
## Nagelkerke R2:  0.1228002
## % pres/err predicted correctly: -732.0185
## % of predictable range [ (model-null)/(1-null) ]:  0.06101707
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stiml
##      7.92160      -0.39942      0.19903      -2.13277      -0.01685      0
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4156 Residual
## Null Deviance:      2792
## Residual Deviance: 2531 AIC: 2543
## log likelihood: -1265.408
## Nagelkerke R2:  0.1244467
## % pres/err predicted correctly: -731.9963
## % of predictable range [ (model-null)/(1-null) ]:  0.0610456
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      4.62967      0.04285      -0.78698
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4159 Residual
## Null Deviance:      2792
## Residual Deviance: 2541 AIC: 2547
## log likelihood: -1270.677
## Nagelkerke R2:  0.1195748
## % pres/err predicted correctly: -733.2172
## % of predictable range [ (model-null)/(1-null) ]:  0.05948159
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos  stimlen:pos

```

```

##      5.94960      -0.27888      -0.76920      0.05015
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4158 Residual
## Null Deviance:      2792
## Residual Deviance: 2545 AIC: 2553
## log likelihood: -1272.341
## Nagelkerke R2:  0.1180343
## % pres/err predicted correctly: -732.6406
## % of predictable range [ (model-null)/(1-null) ]:  0.06022021
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      4.21141      -0.07499      -0.33012
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4159 Residual
## Null Deviance:      2792
## Residual Deviance: 2553 AIC: 2559
## log likelihood: -1276.35
## Nagelkerke R2:  0.1143162
## % pres/err predicted correctly: -732.0908
## % of predictable range [ (model-null)/(1-null) ]:  0.06092454
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.7424      -0.3579
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4160 Residual
## Null Deviance:      2792
## Residual Deviance: 2556 AIC: 2560
## log likelihood: -1278.229
## Nagelkerke R2:  0.1125705
## % pres/err predicted correctly: -732.9789
## % of predictable range [ (model-null)/(1-null) ]:  0.05978689
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.323      -0.274
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4160 Residual

```



```
## Null Deviance:      2792
## Residual Deviance: 2716 AIC: 2720
## log likelihood: -1358.246
## Nagelkerke R2: 0.03677238
## % pres/err predicted correctly: -766.2441
## % of predictable range [ (model-null)/(1-null) ]: 0.01717481
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.145
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4161 Residual
## Null Deviance:      2792
## Residual Deviance: 2792 AIC: 2794
## log likelihood: -1395.984
## Nagelkerke R2: 2.271729e-16
## % pres/err predicted correctly: -779.6516
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                             "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + I(pos^2)	2542.381	0.000000	0.000000	0.000000	0.280784	1228062382702	-	-	NA	0.0477434	NA
+ pos							0.1043417	0.7971613			
preserved ~ stimlen * (I(pos^2)	2542.817	0.435615	0.8042799	0.1247209	0.2444679	21596	-	-	0.1517050	0.1990280	-
+ pos)							0.3994156	0.1327732		0.0168531	
preserved ~ I(pos^2) + pos	2547.354	0.973632	0.0831704	0.0439206	0.1957486	29673	NA	-	NA	0.0428527	NA
								0.7869789			
preserved ~ stimlen * pos	2552.681	0.300160	0.0057989	0.0030623	0.1803439	49597	-	-	0.0501452	NA	NA
							0.2788770	0.17691981			

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~	2558.699	0.318117	0.002861	0.000150	0.114314	2211413	-	-	NA	NA
stimlen + pos							0.074986	0.23301232		
preserved ~ pos	2560.458	0.077126	0.001187	0.000062	0.112573	5742386	NA	-	NA	NA
								0.3578735		
preserved ~	2720.492	278.1114	29700000	0.000000	0.003677	24322833	-	NA	NA	NA
stimlen							0.2739542			
preserved ~ 1	2793.962	51.587521	1000000	0.000000	0.000000	20145448	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen + I(pos^2) + pos"
```

```
print(BestLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept)      stimlen      I(pos^2)          pos
```

```
##      5.38270      -0.10434      0.04774      -0.79716
```

```
##
```

```
## Degrees of Freedom: 4161 Total (i.e. Null); 4158 Residual
```

```
## Null Deviance: 2792
```

```
## Residual Deviance: 2534 AIC: 2542
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
```

```
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
```

```
## # Groups:   stimlen [7]
```

```
## stimlen  `1`  `2`  `3`  `4`  `5`  `6`  `7`  `8`  `9`
```

```
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1      4 0.985 0.972 0.953 NA      NA      NA      NA      NA      NA
```

```
## 2      5 0.984 0.969 0.948 0.920 NA      NA      NA      NA      NA
```

```
## 3      6 0.982 0.966 0.942 0.911 0.877 NA      NA      NA      NA
```

```
## 4      7 0.980 0.963 0.936 0.903 0.865 0.830 NA      NA      NA
```

```
## 5      8 0.978 0.959 0.930 0.893 0.853 0.815 0.787 NA      NA
```

```
## 6      9 0.976 0.954 0.923 0.883 0.839 0.799 0.769 0.754 NA
```

```
## 7     10 0.973 0.950 0.915 0.872 0.824 0.782 0.750 0.734 0.737
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

```

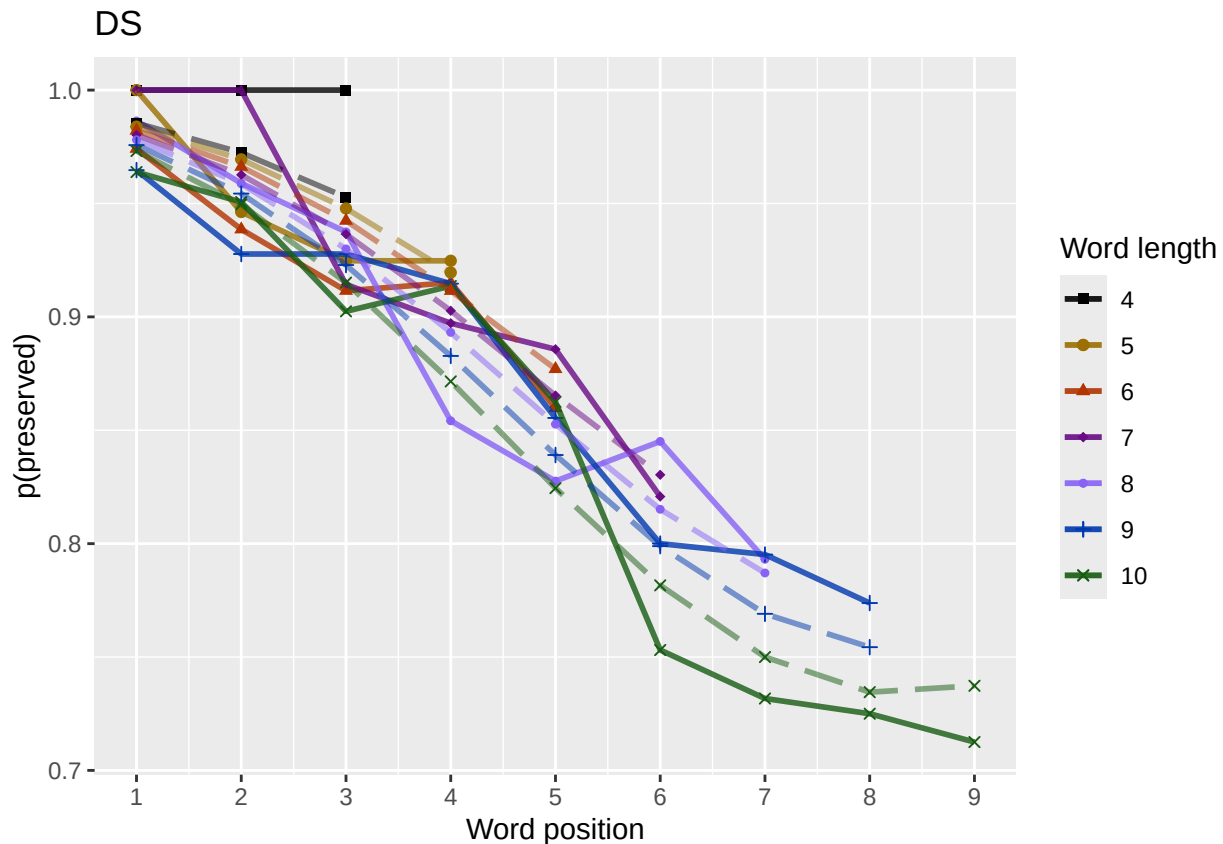
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen))) + ggtitle(pas

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num <- 0

```

```

prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      78   695

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 78 / 695 = 11.22 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##    5.33711    -0.11550     0.06118    -0.69560
##
## Degrees of Freedom: 3935 Total (i.e. Null);  3932 Residual
## Null Deviance:      1640
## Residual Deviance: 1593  AIC: 1601
## log likelihood:  -796.4063
## Nagelkerke R2:   0.03463554
## % pres/err predicted correctly:  -392.2989
## % of predictable range [ (model-null)/(1-null) ]:  0.01079643
## *****

```

```

## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos stimlen:I(pos^2) stimlen:I(pos^2)
## 7.44377 -0.36984 0.19322 -1.84647 -0.01528 0
##
## Degrees of Freedom: 3935 Total (i.e. Null); 3930 Residual
## Null Deviance: 1640
## Residual Deviance: 1591 AIC: 1603
## log likelihood: -795.5151
## Nagelkerke R2: 0.03594875
## % pres/err predicted correctly: -392.3221
## % of predictable range [ (model-null)/(1-null) ]: 0.01073816
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 4.49239 0.05532 -0.67979
##
## Degrees of Freedom: 3935 Total (i.e. Null); 3933 Residual
## Null Deviance: 1640
## Residual Deviance: 1598 AIC: 1604
## log likelihood: -799.2206
## Nagelkerke R2: 0.0304846
## % pres/err predicted correctly: -392.8127
## % of predictable range [ (model-null)/(1-null) ]: 0.009504033
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos stimlen:pos
## 5.98049 -0.30510 -0.68033 0.06185
##
## Degrees of Freedom: 3935 Total (i.e. Null); 3932 Residual
## Null Deviance: 1640
## Residual Deviance: 1601 AIC: 1609
## log likelihood: -800.6166
## Nagelkerke R2: 0.02842323
## % pres/err predicted correctly: -393.1353
## % of predictable range [ (model-null)/(1-null) ]: 0.008692695
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      4.12293      -0.08463      -0.14602
##
## Degrees of Freedom: 3935 Total (i.e. Null);  3933 Residual
## Null Deviance:      1640
## Residual Deviance: 1608  AIC: 1614
## log likelihood:  -804.0048
## Nagelkerke R2:  0.02341441
## % pres/err predicted correctly:  -393.5638
## % of predictable range [ (model-null)/(1-null) ]:  0.007615009
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.577      -0.174
##
## Degrees of Freedom: 3935 Total (i.e. Null);  3934 Residual
## Null Deviance:      1640
## Residual Deviance: 1611  AIC: 1615
## log likelihood:  -805.5496
## Nagelkerke R2:  0.02112778
## % pres/err predicted correctly:  -393.8503
## % of predictable range [ (model-null)/(1-null) ]:  0.006894545
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.1254      -0.1602
##
## Degrees of Freedom: 3935 Total (i.e. Null);  3934 Residual
## Null Deviance:      1640
## Residual Deviance: 1625  AIC: 1629
## log likelihood:  -812.7043
## Nagelkerke R2:  0.01051393
## % pres/err predicted correctly:  -395.2327
## % of predictable range [ (model-null)/(1-null) ]:  0.003417501
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```
## (Intercept)
##      2.876
##
## Degrees of Freedom: 3935 Total (i.e. Null);  3935 Residual
## Null Deviance:      1640
## Residual Deviance: 1640  AIC: 1642
## log likelihood:  -819.7662
## Nagelkerke R2:  -6.517669e-16
## % pres/err predicted correctly:  -396.5915
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + I(pos^2) + pos	1600.813	0.000000	1.000000	0.06622955	0.03463553	37112	-	-	NA	0.0611776 NA
							0.1154973	0.6956048		
preserved ~ stimlen * (I(pos^2) + pos)	1603.032	2.217651	0.329946	0.22185209	0.03594884	43765	-	-	0.1360040	0.1932179 -
							0.3698428	0.464742		0.0152847
preserved ~ I(pos^2) + pos	1604.443	3.628510	0.162959	0.1079270	0.03048464	492395	NA	-	NA	0.0553178 NA
								0.6797852		
preserved ~ stimlen * pos	1609.233	8.420662	0.014840	0.0098294	0.02842329	80488	-	-	0.0618547	NA NA
							0.3051001	0.6803285		
preserved ~ stimlen + pos	1614.010	3.196930	0.001362	0.0009023	0.02341441	22932	-	-	NA NA	NA
							0.0846270	0.1460171		
preserved ~ pos	1615.090	4.286562	0.000790	0.0005233	0.02112785	76531	NA	-	NA NA	NA
								0.1740462		
preserved ~ stimlen	1629.408	28.596035	0.000000	0.000000	0.01051391	125432	-	NA	NA NA	NA
							0.1602368			
preserved ~ 1	1641.532	40.719686	0.000000	0.000000	0.000000	20875983	NA	NA	NA NA	NA

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.986 0.977 0.966 NA      NA      NA      NA      NA      NA
## 2      5 0.984 0.974 0.962 0.951 NA      NA      NA      NA      NA
## 3      6 0.982 0.971 0.957 0.945 0.937 NA      NA      NA      NA
## 4      7 0.980 0.967 0.952 0.938 0.930 0.928 NA      NA      NA
## 5      8 0.978 0.963 0.947 0.931 0.922 0.920 0.927 NA      NA
## 6      9 0.975 0.959 0.941 0.924 0.913 0.911 0.919 0.934 NA
## 7     10 0.972 0.954 0.934 0.915 0.903 0.901 0.910 0.926 0.947
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
```

```
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
```

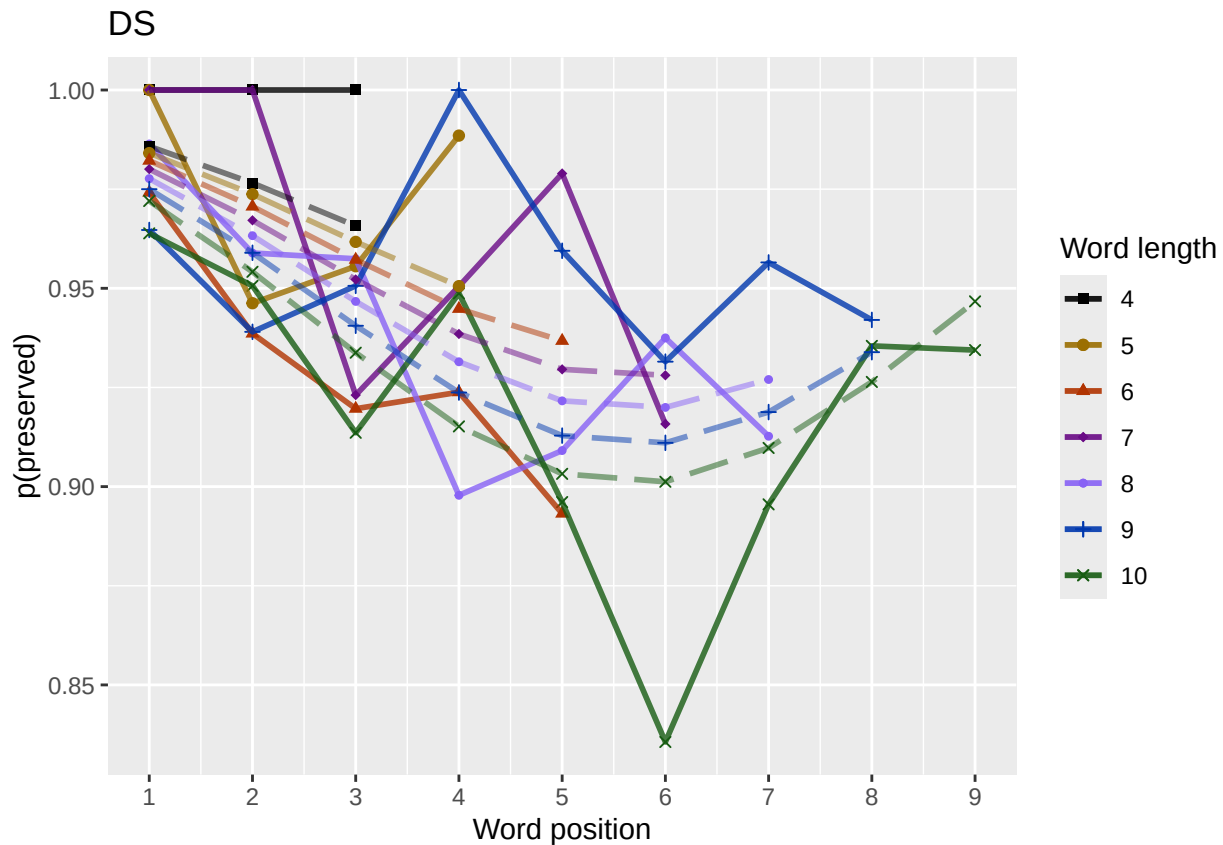
```
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted
```

```
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.68 - 1.03"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.007484341
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] -0.0292137
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
    2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "Average upward change after U minimum"
## [1] 0.002813343

if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "differences from left max to min for each row: "
## [1] 0.03272199 0.06433255 0.10512956 0.14986671 0.19103264 0.22139306 0.23864037
## [1] "differences from min to right max for each row: "
## [1] 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000 0.002813343
## [1] " "
## [1] "row with biggest return upward"
## [1] 7
## [1] " "
## [1] "downward distance for row with the largest upward value"
## [1] 0.2386404
## [1] 0.002813343
## [1] " "
## [1] "Return upward as a proportion of the downward distance:"
## [1] 0.01178905

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",

```

```

    "preserved ~ stimlen*pos",
    "preserved ~ I(pos^2)+pos",
    "preserved ~ stimlen + I(pos^2) + pos",
    "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 13
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos      log_freq
##      5.27175      -0.09007      0.04744      -0.79449      0.04539
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4157 Residual
## Null Deviance:      2792
## Residual Deviance: 2532 AIC: 2542
## log likelihood: -1265.937
## Nagelkerke R2: 0.123958
## % pres/err predicted correctly: -731.5724
## % of predictable range [ (model-null)/(1-null) ]: 0.06158851
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      5.38270      -0.10434      0.04774      -0.79716
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4158 Residual
## Null Deviance:      2792
## Residual Deviance: 2534 AIC: 2542
## log likelihood: -1267.191
## Nagelkerke R2: 0.1228002
## % pres/err predicted correctly: -732.0185
## % of predictable range [ (model-null)/(1-null) ]: 0.06101707
## *****
## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      log_freq      I(pos^2)          pos      stimlen:log
##      5.22245      -0.08774      0.21610      0.04668      -0.78659      -0
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4156 Residual
## Null Deviance:      2792

```

```

## Residual Deviance: 2531 AIC: 2543
## log likelihood: -1265.32
## Nagelkerke R2: 0.1245284
## % pres/err predicted correctly: -731.6106
## % of predictable range [ (model-null)/(1-null) ]: 0.06153961
## *****
## model index: 21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos stimlen:I(pos^2) stimlen:I(pos^2)
## 7.92160 -0.39942 0.19903 -2.13277 -0.01685 0
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4156 Residual
## Null Deviance: 2792
## Residual Deviance: 2531 AIC: 2543
## log likelihood: -1265.408
## Nagelkerke R2: 0.1244467
## % pres/err predicted correctly: -731.9963
## % of predictable range [ (model-null)/(1-null) ]: 0.0610456
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq I(pos^2)
## 5.2249075 -0.0839552 0.0461897 -0.7885589 0.0994888 0
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4155 Residual
## Null Deviance: 2792
## Residual Deviance: 2531 AIC: 2545
## log likelihood: -1265.477
## Nagelkerke R2: 0.1243837
## % pres/err predicted correctly: -731.5421
## % of predictable range [ (model-null)/(1-null) ]: 0.06162742
## *****
## model index: 10
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stimlen:I(pos^2)
## 5.2121451 -0.0850738 0.2178214 0.0463140 -0.7865609 0
## log_freq:pos
## -0.0080880
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4154 Residual
## Null Deviance: 2792
## Residual Deviance: 2530 AIC: 2546

```

```

## log likelihood: -1265.196
## Nagelkerke R2: 0.1246428
## % pres/err predicted correctly: -731.5853
## % of predictable range [ (model-null)/(1-null) ]: 0.06157202
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos log_freq I(pos^2):log_freq
## 4.6136316 0.0420156 -0.7776891 0.1326211 -0.0008629
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4156 Residual
## Null Deviance: 2792
## Residual Deviance: 2535 AIC: 2547
## log likelihood: -1267.551
## Nagelkerke R2: 0.1224673
## % pres/err predicted correctly: -732.2786
## % of predictable range [ (model-null)/(1-null) ]: 0.06068392
## *****
## model index: 19
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 4.62967 0.04285 -0.78698
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4159 Residual
## Null Deviance: 2792
## Residual Deviance: 2541 AIC: 2547
## log likelihood: -1270.677
## Nagelkerke R2: 0.1195748
## % pres/err predicted correctly: -733.2172
## % of predictable range [ (model-null)/(1-null) ]: 0.05948159
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos stimlen:pos
## 5.94960 -0.27888 -0.76920 0.05015
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4158 Residual
## Null Deviance: 2792
## Residual Deviance: 2545 AIC: 2553
## log likelihood: -1272.341
## Nagelkerke R2: 0.1180343
## % pres/err predicted correctly: -732.6406

```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.06022021
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq pos:log_freq
##      3.74767      -0.35800      0.15750      -0.01982
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4158 Residual
## Null Deviance: 2792
## Residual Deviance: 2550 AIC: 2558
## log likelihood: -1274.907
## Nagelkerke R2: 0.1156548
## % pres/err predicted correctly: -732.1421
## % of predictable range [ (model-null)/(1-null) ]: 0.06085874
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq
##      4.10252      -0.06007      -0.33055      0.04780
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4158 Residual
## Null Deviance: 2792
## Residual Deviance: 2550 AIC: 2558
## log likelihood: -1274.966
## Nagelkerke R2: 0.1156006
## % pres/err predicted correctly: -731.6551
## % of predictable range [ (model-null)/(1-null) ]: 0.06148265
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      log_freq          pos stimlen:log_freq
##      4.06520      -0.05760      0.25118      -0.33028      -0.02468
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4157 Residual
## Null Deviance: 2792
## Residual Deviance: 2548 AIC: 2558
## log likelihood: -1274.034
## Nagelkerke R2: 0.1164645
## % pres/err predicted correctly: -731.7184
## % of predictable range [ (model-null)/(1-null) ]: 0.06140153
## *****
## model index: 7

```



```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq pos:log_freq
##      4.07429      -0.05216      -0.33893      0.13590      -0.01725
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4157 Residual
## Null Deviance:      2792
## Residual Deviance: 2548  AIC: 2558
## log likelihood:  -1274.07
## Nagelkerke R2:  0.1164312
## % pres/err predicted correctly:  -731.6609
## % of predictable range [ (model-null)/(1-null) ]:  0.06147515
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq
##      3.72205      -0.35147      0.05769
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4159 Residual
## Null Deviance:      2792
## Residual Deviance: 2552  AIC: 2558
## log likelihood:  -1276.104
## Nagelkerke R2:  0.1145439
## % pres/err predicted correctly:  -732.2326
## % of predictable range [ (model-null)/(1-null) ]:  0.06074284
## *****
## model index:  17
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      4.21141      -0.07499      -0.33012
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4159 Residual
## Null Deviance:      2792
## Residual Deviance: 2553  AIC: 2559
## log likelihood:  -1276.35
## Nagelkerke R2:  0.1143162
## % pres/err predicted correctly:  -732.0908
## % of predictable range [ (model-null)/(1-null) ]:  0.06092454
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)

```

```

##
## Coefficients:
##      (Intercept)      stimlen      log_freq      pos  stimlen:log_freq      log_fr
##      4.06144      -0.05373      0.24507      -0.33560      -0.01687      -0
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4156 Residual
## Null Deviance:      2792
## Residual Deviance: 2547  AIC: 2559
## log likelihood: -1273.738
## Nagelkerke R2:  0.1167388
## % pres/err predicted correctly: -731.6951
## % of predictable range [ (model-null)/(1-null) ]:  0.06143141
## *****
## model index:  16
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      pos
##      3.7424      -0.3579
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4160 Residual
## Null Deviance:      2792
## Residual Deviance: 2556  AIC: 2560
## log likelihood: -1278.229
## Nagelkerke R2:  0.1125705
## % pres/err predicted correctly: -732.9789
## % of predictable range [ (model-null)/(1-null) ]:  0.05978689
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq  stimlen:log_freq
##      4.18502      -0.25782      0.25404      -0.02534
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4158 Residual
## Null Deviance:      2792
## Residual Deviance: 2712  AIC: 2720
## log likelihood: -1355.978
## Nagelkerke R2:  0.03896153
## % pres/err predicted correctly: -765.9149
## % of predictable range [ (model-null)/(1-null) ]:  0.01759646
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq

```

```
##      4.22235      -0.26043      0.04403
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4159 Residual
## Null Deviance:      2792
## Residual Deviance: 2714 AIC: 2720
## log likelihood: -1357.006
## Nagelkerke R2:  0.03796987
## % pres/err predicted correctly: -765.8386
## % of predictable range [ (model-null)/(1-null) ]:  0.01769417
## *****
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.323      -0.274
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4160 Residual
## Null Deviance:      2792
## Residual Deviance: 2716 AIC: 2720
## log likelihood: -1358.246
## Nagelkerke R2:  0.03677238
## % pres/err predicted correctly: -766.2441
## % of predictable range [ (model-null)/(1-null) ]:  0.01717481
## *****
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.145
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4161 Residual
## Null Deviance:      2792
## Residual Deviance: 2792 AIC: 2794
## log likelihood: -1395.984
## Nagelkerke R2:  2.271729e-16
## % pres/err predicted correctly: -779.6516
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                           AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2
```

```
FLPAICSummary <- merge(FLPAICSummary, FLPRes$CoefficientValues,
                        by='row.names', sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_freq_len_pos_model_summary.csv"), row.names = FALSE)

kable(FLPAICSummary)
```

Model	AIC	Delta AIC	C _w	Nag R ²	(Intercept)	log_freq	stimlen	pos	log_freq	I(pos^2)	log_freq	I(pos^2)	log_freq	I(pos^2)	log_freq	I(pos^2)
preserved ~ stimlen + I(pos^2) + pos + log_freq	2541.875	0.000000	0.981	0.612	0.393	0.0453029	-	NA	NA	0.047411	NA	NA	NA	NA	NA	
						0.0900721	0.7944905									
preserved ~ stimlen + I(pos^2) + pos	2542.350	0.475	0.976	0.612	0.382	0.0453029	NA	NA	-	NA	NA	0.047411	NA	NA	NA	
						0.1043417	0.7971613									
preserved ~ stimlen * log_freq + I(pos^2) + pos	2542.646	0.771	0.968	0.612	0.366	0.2160989	-	NA	NA	0.0466849	NA	NA	NA	NA	NA	
						0.0877408	0.0206312	0.2865895								
preserved ~ stimlen * (I(pos^2) + pos)	2542.891	1.016	0.964	0.617	0.344	0.21596	NA	NA	-	NA	NA	0.1990210	NA	0.1517056		
						0.3994156	2.1327732							0.0168531		
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	2544.353	2.478	0.938	0.616	0.324	0.0994888	-	-	NA	0.0461897	NA	NA	NA	NA	NA	
						0.0839552	0.7885396	0.0007115								
preserved ~ stimlen * log_freq + (I(pos^2) + pos) *	2546.492	4.617	0.884	0.599	0.285	0.2178214	-	NA	-	0.0463140	2.17e-05	NA	NA	NA	NA	
						0.0850738	0.0158560	0.00865609								
preserved ~ (I(pos^2) + pos) * log_freq	2547.122	5.247	0.876	0.602	0.267	0.1326211	-	-	NA	0.0420156	NA	NA	NA	NA	NA	
							0.7776890	0.1657								
preserved ~ I(pos^2) + pos	2547.343	5.468	0.864	0.608	0.257	0.129671	NA	NA	-	NA	NA	0.0428527	NA	NA	NA	
							0.7869789									
preserved ~ stimlen * pos	2552.688	10.813	0.754	0.021	0.693	0.49597	NA	NA	-	NA	NA	NA	NA	0.0501452		
						0.2788774	0.7691981									
preserved ~ pos *	2557.814	15.939	0.300	0.083	0.580	0.1574068	-	-	NA	NA	NA	NA	NA	NA	NA	
							0.3580005	0.98180								

Model	AIC	Delta AIC	AICw	NagR ²	(Intercept)	stimlen	log_freq	pos	log_freq	I(pos^2)	pos^2	log_freq	I(pos^2)	pos	log_freq	I(pos^2)
preserved ~ stimlen + pos + log_freq	2557.13	0.0561	0.0032	0.0925	0.0477	0.048	-	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen *	2558.16	0.0332	0.0030	0.0856	0.0575	0.045	-	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen + pos * log_freq	2558.16	0.0510	0.0030	0.0826	0.0521	0.043	-	-	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ pos + log_freq	2558.16	0.0334	0.0030	0.0814	0.0576	0.044	-	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen + pos	2558.16	0.0524	0.0032	0.0626	0.0749	0.041	-	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen *	2559.17	0.0623	0.0030	0.0604	0.0537	0.041	-	NA	-	NA	NA	NA	NA	NA	NA	NA
preserved ~ pos	2560.15	0.0832	0.0030	0.0262	0.0379	0.038	-	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen *	2719.17	0.0808	0.0000	0.0389	0.2540	0.015	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen + log_freq	2720.17	0.1369	0.0000	0.0379	0.0440	0.022	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen	2720.17	0.0754	0.0000	0.0367	0.2739	0.022	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ 1	2793.25	0.0936	0.0000	0.0002	0.0454	0.015	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen + I(pos^2) + pos + log_freq"
```

```
print(BestFLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos      log_freq
##      5.27175      -0.09007      0.04744      -0.79449      0.04539
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4157 Residual
## Null Deviance:      2792
## Residual Deviance: 2532  AIC: 2542

# do a median split on frequency to plot hf/lr effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
```

```

PosDat$freq_bin[PosDat$log_freq < median_freq]<-"lf"

PosDat$FLPFitted<-fitted(BestFLPModel)

HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]

HF_Plot <- plot_len_pos_obs_predicted(HFDat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

LF_Plot <- plot_len_pos_obs_predicted(LFDat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted")

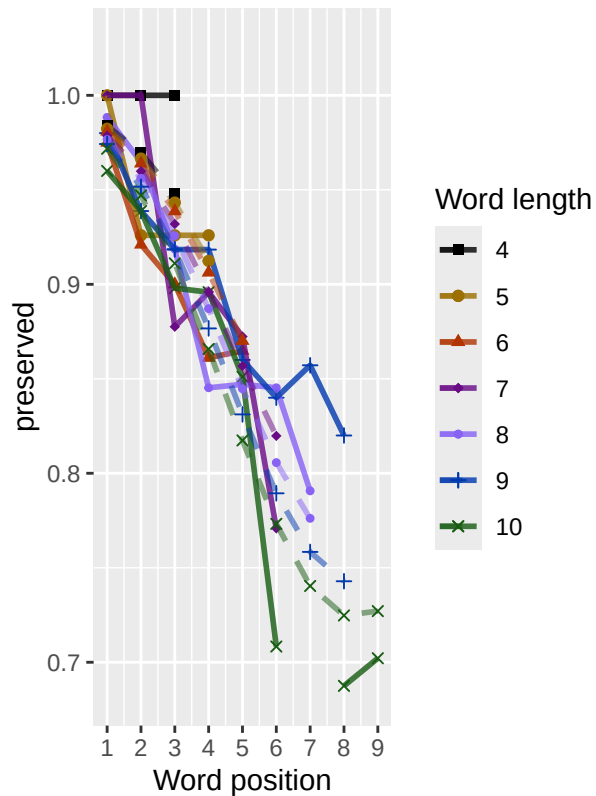
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)

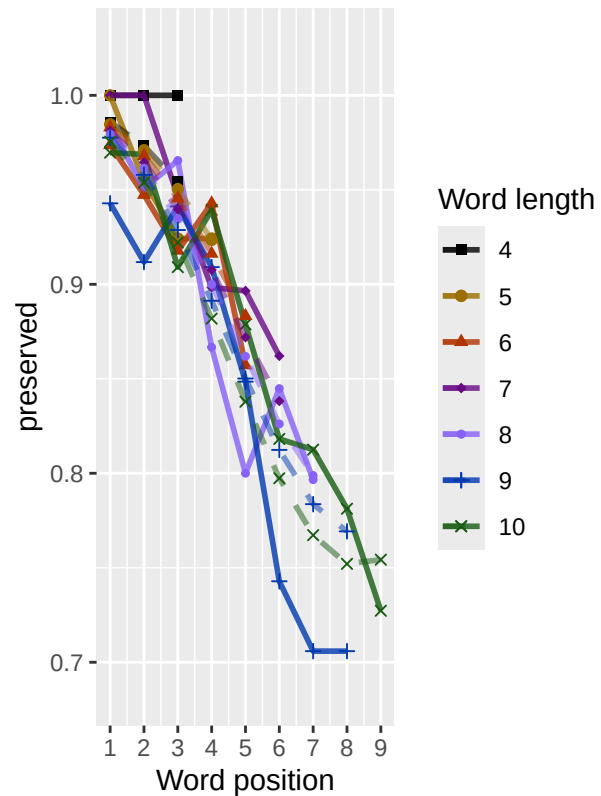
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)

```

DS – no_frac – Low frequency



DS – no_frac – High frequency



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.896      -2.068
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4160 Residual
## Null Deviance:      2792
## Residual Deviance: 1899 AIC: 1903
## log likelihood: -949.6046
## Nagelkerke R2:  0.395032
## % pres/err predicted correctly: -489.9273
```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.3711313
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 4.62967 0.04285 -0.78698
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4159 Residual
## Null Deviance: 2792
## Residual Deviance: 2541 AIC: 2547
## log likelihood: -1270.677
## Nagelkerke R2: 0.1195748
## % pres/err predicted correctly: -733.2172
## % of predictable range [ (model-null)/(1-null) ]: 0.05948159
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 3.7424 -0.3579
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4160 Residual
## Null Deviance: 2792
## Residual Deviance: 2556 AIC: 2560
## log likelihood: -1278.229
## Nagelkerke R2: 0.1125705
## % pres/err predicted correctly: -732.9789
## % of predictable range [ (model-null)/(1-null) ]: 0.05978689
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 4.323 -0.274
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4160 Residual
## Null Deviance: 2792
## Residual Deviance: 2716 AIC: 2720
## log likelihood: -1358.246
## Nagelkerke R2: 0.03677238
## % pres/err predicted correctly: -766.2441
## % of predictable range [ (model-null)/(1-null) ]: 0.01717481
## *****
## model index: 6

```



```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.145
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4161 Residual
## Null Deviance:      2792
## Residual Deviance: 2792  AIC: 2794
## log likelihood:  -1395.984
## Nagelkerke R2:  2.271729e-16
## % pres/err predicted correctly:  -779.6516
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.20525      -0.02282
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4160 Residual
## Null Deviance:      2792
## Residual Deviance: 2791  AIC: 2795
## log likelihood:  -1395.564
## Nagelkerke R2:  0.0004136546
## % pres/err predicted correctly:  -779.5129
## % of predictable range [ (model-null)/(1-null) ]:  0.0001777421
## *****

BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1903.2090	0.0000	1	1	0.3950320	0.896421	NA	-	NA	NA	NA
								2.067928			
preserved ~ (I(pos^2) + pos)	2547.3536	44.1454	0	0	0.1195748	0.629673	NA	NA	0.0428527	-	NA
										0.7869789	
preserved ~ pos	2560.4586	57.2489	0	0	0.1125703	0.742386	NA	NA	NA	-	NA
										0.3578735	
preserved ~ stimlen	2720.4931	17.2832	0	0	0.0367724	0.322833	NA	NA	NA	NA	-
											0.2739542
preserved ~ 1	2793.9698	90.7593	0	0	0.0000000	0.145448	NA	NA	NA	NA	NA
preserved ~ CumPres	2795.1278	91.9178	0	0	0.0004132	0.205245	-	NA	NA	NA	NA
							0.022822				

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                        family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                 rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
  BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
  BestMEModelRndDF <- rbind(BestMEModelRndDF,
                            data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
  BestMEModelRndDF <- rbind(BestMEModelRndDF,
                            data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

  write.csv(BestMEModelRndDF,
            paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                  "_best_main_effects_model_with_random_cum_term.csv"),
            row.names = FALSE)
}

```

```

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                       RunLabel,"_",CurTask,

```

```

kable(syll_component_summary)                                     "_syllable_component_summary.csv"),row.names = FALSE)

```

syll_component	MeanPres	N
1	0.8913444	543
O	0.8841401	1942
P	1.0000000	36
S	0.8683128	243
V	0.9141631	1398

```

# main effects models for data without satellite positions

```

```

keep_components = c("0","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)

```

```

## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.904      -2.327
##
## Degrees of Freedom: 3882 Total (i.e. Null);  3881 Residual
## Null Deviance:      2593
## Residual Deviance: 1742  AIC: 1746
## log likelihood:  -871.0812
## Nagelkerke R2:  0.4038889
## % pres/err predicted correctly:  -445.5705
## % of predictable range [ (model-null)/(1-null) ]:  0.3831329
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      4.62010      0.04102     -0.77074
##
## Degrees of Freedom: 3882 Total (i.e. Null);  3880 Residual
## Null Deviance:      2593
## Residual Deviance: 2361  AIC: 2367

```

```

## log likelihood: -1180.684
## Nagelkerke R2: 0.1188193
## % pres/err predicted correctly: -680.2386
## % of predictable range [ (model-null)/(1-null) ]: 0.05897573
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.765      -0.359
##
## Degrees of Freedom: 3882 Total (i.e. Null); 3881 Residual
## Null Deviance: 2593
## Residual Deviance: 2374 AIC: 2378
## log likelihood: -1187.065
## Nagelkerke R2: 0.1124527
## % pres/err predicted correctly: -679.7857
## % of predictable range [ (model-null)/(1-null) ]: 0.05960127
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.258      -0.265
##
## Degrees of Freedom: 3882 Total (i.e. Null); 3881 Residual
## Null Deviance: 2593
## Residual Deviance: 2527 AIC: 2531
## log likelihood: -1263.483
## Nagelkerke R2: 0.03455749
## % pres/err predicted correctly: -711.26
## % of predictable range [ (model-null)/(1-null) ]: 0.01612454
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.153
##
## Degrees of Freedom: 3882 Total (i.e. Null); 3882 Residual
## Null Deviance: 2593
## Residual Deviance: 2593 AIC: 2595
## log likelihood: -1296.445
## Nagelkerke R2: 4.558118e-16
## % pres/err predicted correctly: -722.933

```

```
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.18724      -0.01403
##
## Degrees of Freedom: 3882 Total (i.e. Null); 3881 Residual
## Null Deviance:      2593
## Residual Deviance: 2593 AIC: 2597
## log likelihood: -1296.311
## Nagelkerke R2: 0.0001420903
## % pres/err predicted correctly: -722.8888
## % of predictable range [ (model-null)/(1-null) ]: 6.112238e-05
## *****
```

```
write.csv(SimpSyllMEAICSummary,
          paste0(TablesDir, CurPat, "_", RunLabel, "_",
                  CurTask,
                  "_OV1_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1746.162	0.0000	1	1	0.4038882	2.903926	NA	-	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2367.366	21.2060	0	0	0.1188193	1.620098	NA	NA	0.0410211	-	NA
preserved ~ pos	2378.136	31.9676	0	0	0.1124523	1.764881	NA	NA	NA	-	NA
preserved ~ stimlen	2530.966	784.8035	0	0	0.0345574	1.257557	NA	NA	NA	NA	-
preserved ~ 1	2594.898	48.7278	0	0	0.0000000	0.153085	NA	NA	NA	NA	NA
preserved ~ CumPres	2596.621	50.4590	0	0	0.0001422	1.187240	-	NA	NA	NA	NA
							0.0140327				

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0", "V")
OVData <- PosDat[PosDat$syll_component %in% keep_components,]
OVData <- OVData %>% select(stim_number,
                           stimlen, stim, pos,
                           preserved, syll_component)
OVData$CumPres <- CalcCumPres(OVData)
OVData$CumErr <- CalcCumErrFromPreserved(OVData)

SimpSyllMEAICSummary2 <- EvaluateSubsetData(OVData, MEModelEquations)
```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.848      -2.603
##
## Degrees of Freedom: 3339 Total (i.e. Null); 3338 Residual
## Null Deviance:      2219
## Residual Deviance: 1546 AIC: 1550
## log likelihood: -773.2403
## Nagelkerke R2: 0.3759118
## % pres/err predicted correctly: -397.3206
## % of predictable range [ (model-null)/(1-null) ]: 0.3562262
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      4.52682      0.03755      -0.73042
##
## Degrees of Freedom: 3339 Total (i.e. Null); 3337 Residual
## Null Deviance:      2219
## Residual Deviance: 2018 AIC: 2024
## log likelihood: -1008.77
## Nagelkerke R2: 0.1208577
## % pres/err predicted correctly: -580.6959
## % of predictable range [ (model-null)/(1-null) ]: 0.05985136
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.7570      -0.3544
##
## Degrees of Freedom: 3339 Total (i.e. Null); 3338 Residual
## Null Deviance:      2219
## Residual Deviance: 2028 AIC: 2032
## log likelihood: -1013.809
## Nagelkerke R2: 0.1149977
## % pres/err predicted correctly: -580.1467
## % of predictable range [ (model-null)/(1-null) ]: 0.06073891
## *****
## model index: 5
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.1679      -0.2535
##
## Degrees of Freedom: 3339 Total (i.e. Null); 3338 Residual
## Null Deviance:      2219
## Residual Deviance: 2167 AIC: 2171
## log likelihood: -1083.453
## Nagelkerke R2: 0.03217798
## % pres/err predicted correctly: -608.6075
## % of predictable range [ (model-null)/(1-null) ]: 0.01474001
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.27292      -0.05343
##
## Degrees of Freedom: 3339 Total (i.e. Null); 3338 Residual
## Null Deviance:      2219
## Residual Deviance: 2217 AIC: 2221
## log likelihood: -1108.437
## Nagelkerke R2: 0.001615315
## % pres/err predicted correctly: -617.2953
## % of predictable range [ (model-null)/(1-null) ]: 0.0006986243
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.161
##
## Degrees of Freedom: 3339 Total (i.e. Null); 3339 Residual
## Null Deviance:      2219
## Residual Deviance: 2219 AIC: 2221
## log likelihood: -1109.747
## Nagelkerke R2: 2.28686e-16
## % pres/err predicted correctly: -617.7275
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
write.csv(SimpSyllMEAICSummary2,
         paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
               "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)

```

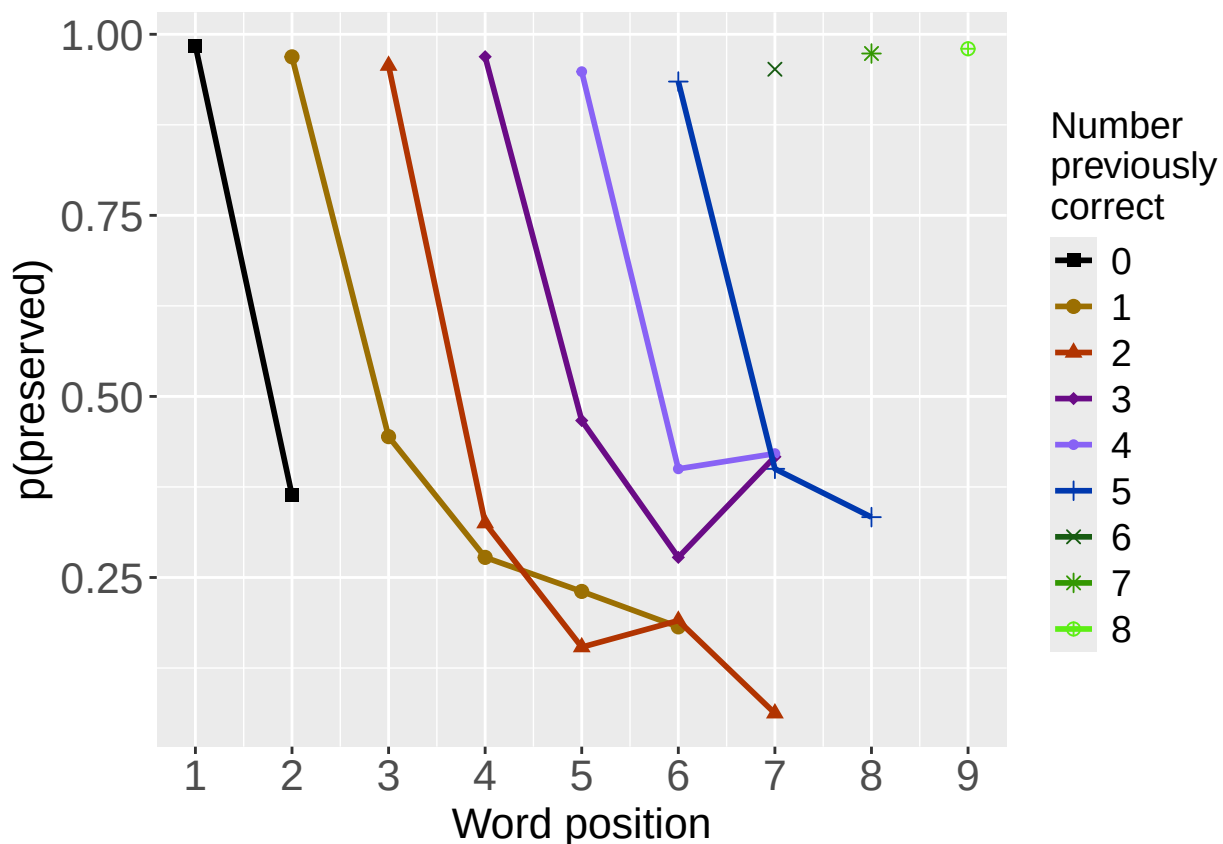
Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1550.4810	0.0000	1	1	0.3759118	8.848474	NA	-	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2023.5394	73.0587	0	0	0.1208574	5.526817	NA	NA	0.037554	-	NA
preserved ~ pos	2031.6184	81.1376	0	0	0.1149973	7.757024	NA	NA	NA	-	NA
preserved ~ stimlen	2170.9066	20.4255	0	0	0.0321780	0.167938	NA	NA	NA	NA	-
preserved ~ CumPres	2220.8746	70.3937	0	0	0.0016153	2.272917	-	NA	NA	NA	NA
preserved ~ 1	2221.4957	71.0139	0	0	0.0000000	0.161155	0.0534347	NA	NA	NA	NA

```
# plot prev err and prev cor plots
```

```
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

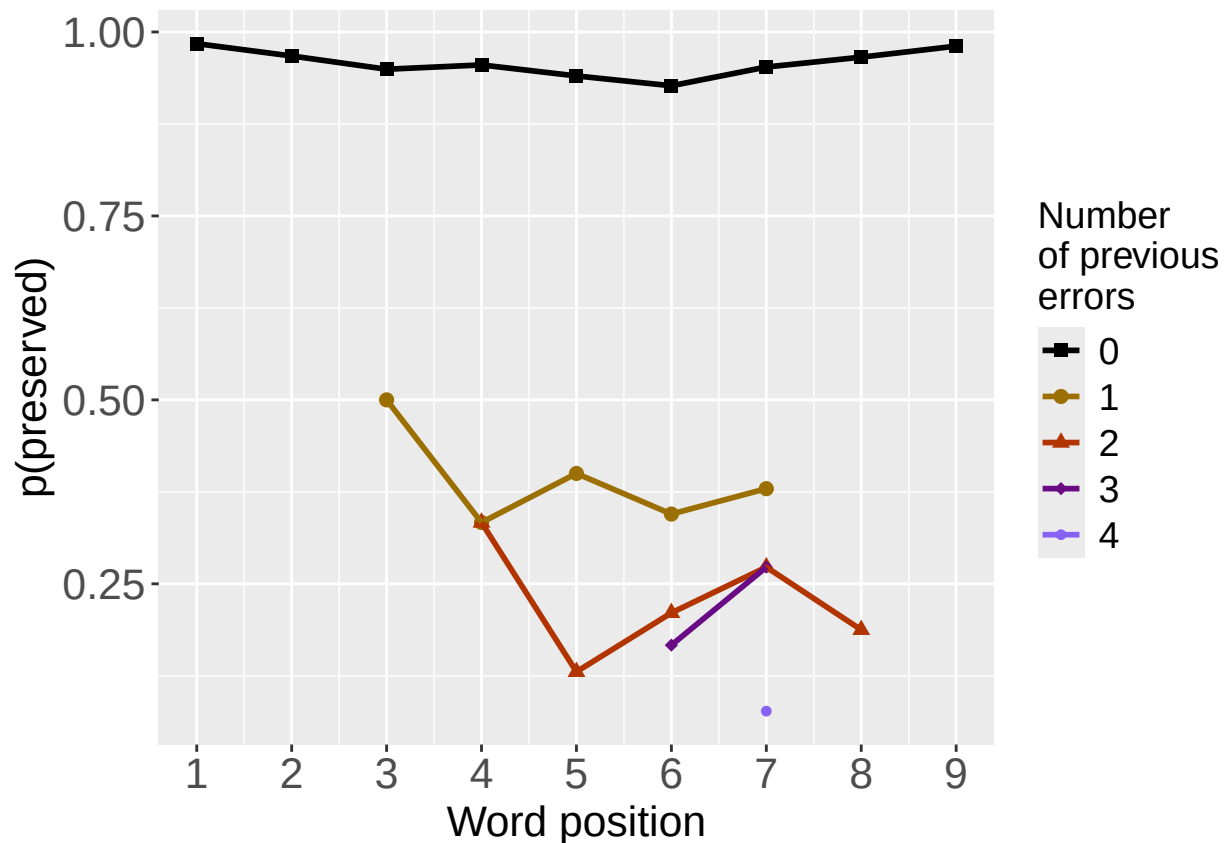
```
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

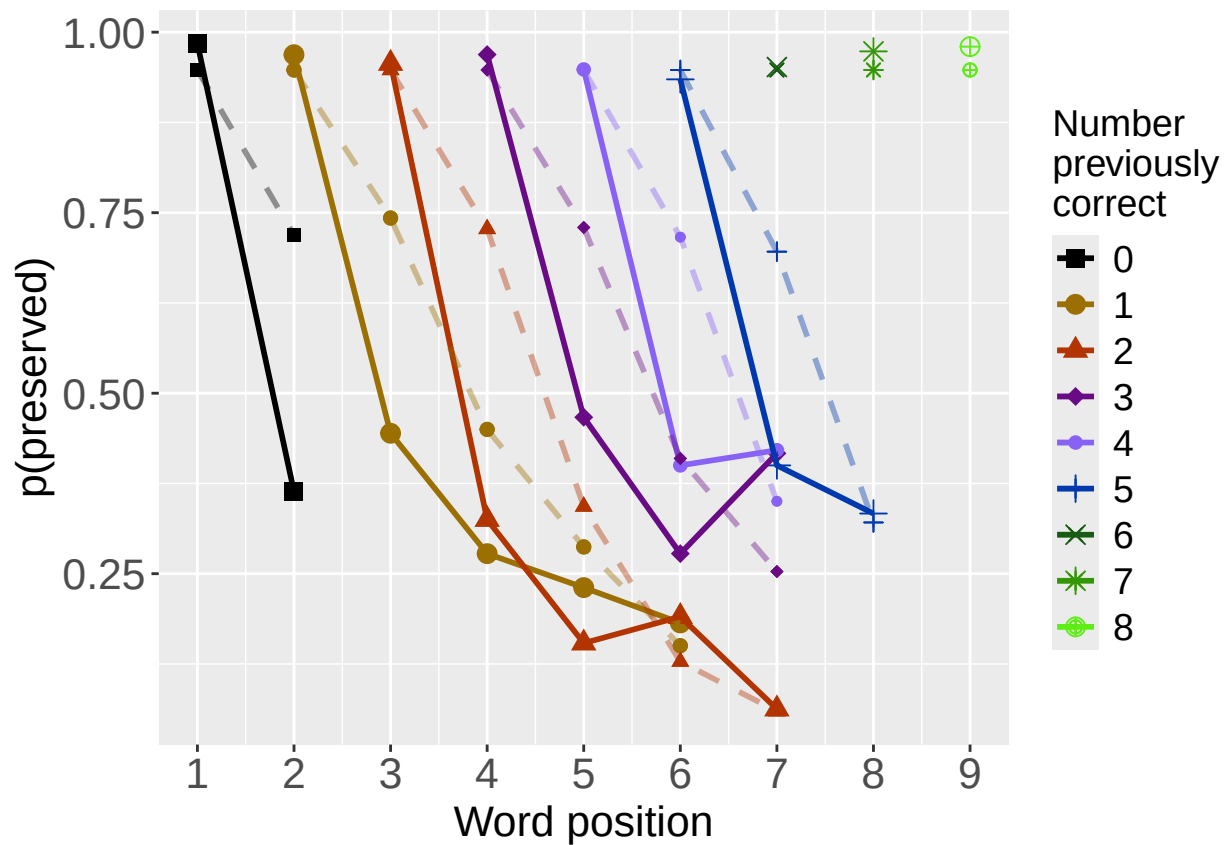
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

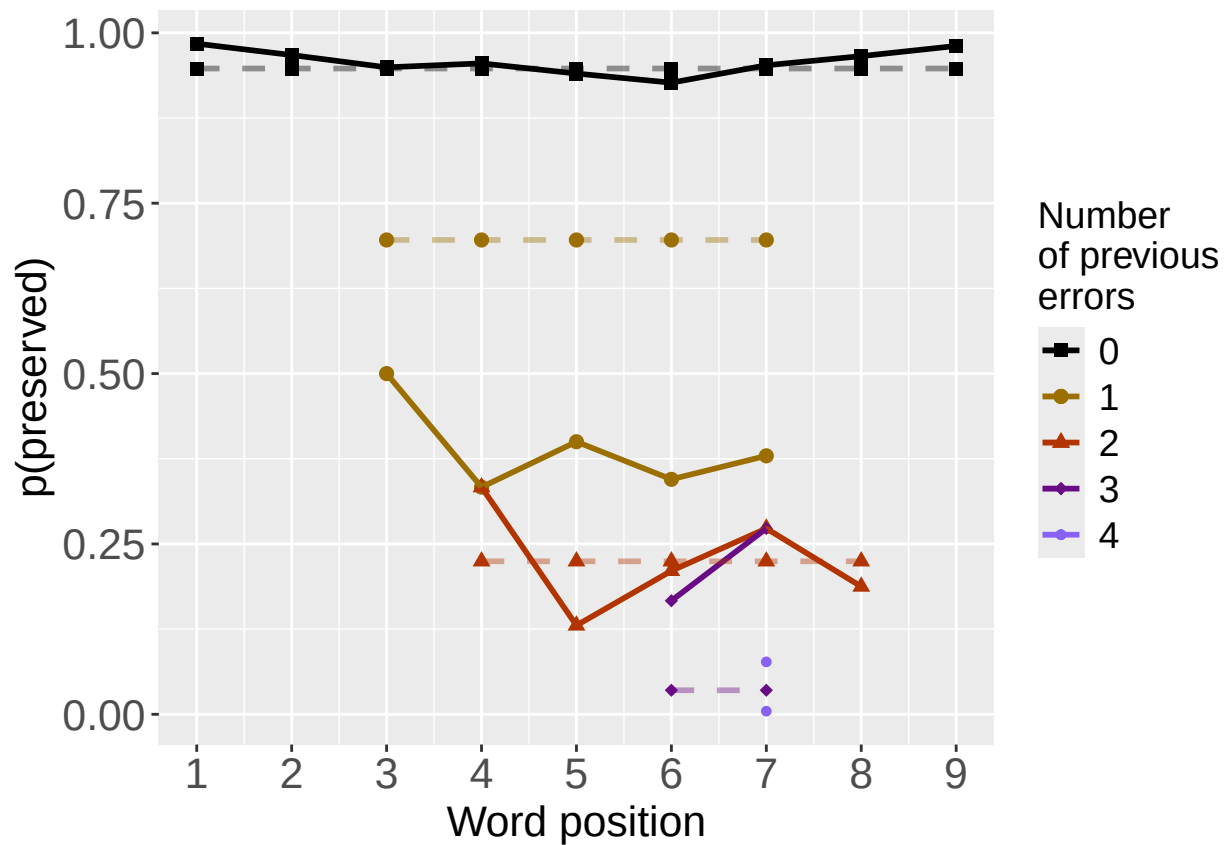
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##    4.69321    -1.93849     0.07776    -0.83667
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4158 Residual
## Null Deviance:      2792
## Residual Deviance: 1855  AIC: 1863
## log likelihood:  -927.6356
## Nagelkerke R2:  0.4123715
## % pres/err predicted correctly:  -484.3936
## % of predictable range [ (model-null)/(1-null) ]:  0.37822
## *****
## model index: 1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.896      -2.068
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4160 Residual
## Null Deviance:      2792
## Residual Deviance: 1899  AIC: 1903
## log likelihood:  -949.6046
## Nagelkerke R2:  0.395032
## % pres/err predicted correctly:  -489.9273
## % of predictable range [ (model-null)/(1-null) ]:  0.3711313
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      4.62967      0.04285      -0.78698
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4159 Residual
## Null Deviance:      2792
## Residual Deviance: 2541  AIC: 2547
## log likelihood:  -1270.677
## Nagelkerke R2:  0.1195748
## % pres/err predicted correctly:  -733.2172
## % of predictable range [ (model-null)/(1-null) ]:  0.05948159
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	1863.271	0.00000	1	1	0.4123715	4.693207	-1.938494	0.0777639	-0.8366701
preserved ~ CumErr	1903.209	39.93799	0	0	0.3950320	2.896421	-2.067928	NA	NA
preserved ~ I(pos^2) + pos	2547.355	684.08339	0	0	0.1195748	4.629673	NA	0.0428527	-0.7869789

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 3.61993 -2.02799 -0.09404
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4159 Residual
## Null Deviance: 2792
## Residual Deviance: 1893 AIC: 1899
## log likelihood: -946.6998
## Nagelkerke R2: 0.3973352
## % pres/err predicted correctly: -489.3937
## % of predictable range [ (model-null)/(1-null) ]: 0.3718149
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 2.896 -2.068
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4160 Residual
## Null Deviance: 2792
## Residual Deviance: 1899 AIC: 1903
## log likelihood: -949.6046
## Nagelkerke R2: 0.395032
## % pres/err predicted correctly: -489.9273
## % of predictable range [ (model-null)/(1-null) ]: 0.3711313
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      4.323      -0.274
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4160 Residual
## Null Deviance:      2792
## Residual Deviance: 2716  AIC: 2720
## log likelihood:  -1358.246
## Nagelkerke R2:  0.03677238
## % pres/err predicted correctly:  -766.2441
## % of predictable range [ (model-null)/(1-null) ]:  0.01717481
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr + stimlen	1899.400	0.000000	1.0000000	0.8704392	0.3973352	3.619925	-	-
preserved ~ CumErr	1903.209	3.809696	0.1488453	0.1295608	0.3950320	2.896421	-	NA
preserved ~ stimlen	2720.492	821.092898	0.0000000	0.0000000	0.0367724	4.322833	NA	-
								0.2739542

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      3.08019      -2.06731      -0.06725
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4159 Residual
## Null Deviance:      2792
## Residual Deviance: 1895  AIC: 1901
## log likelihood:  -947.3508
## Nagelkerke R2:  0.3968193
## % pres/err predicted correctly:  -489.8582
## % of predictable range [ (model-null)/(1-null) ]:  0.3712199
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.896      -2.068
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4160 Residual
## Null Deviance:      2792
## Residual Deviance: 1899  AIC: 1903
## log likelihood:  -949.6046
## Nagelkerke R2:  0.395032
## % pres/err predicted correctly:  -489.9273
## % of predictable range [ (model-null)/(1-null) ]:  0.3711313
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.20525      -0.02282
##
## Degrees of Freedom: 4161 Total (i.e. Null);  4160 Residual
## Null Deviance:      2792
## Residual Deviance: 2791  AIC: 2795
## log likelihood:  -1395.564
## Nagelkerke R2:  0.0004136546
## % pres/err predicted correctly:  -779.5129
## % of predictable range [ (model-null)/(1-null) ]:  0.0001777421
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	1900.702	0.000000	1.0000000	0.7779658	0.3968193	3.080188	- 2.067310	- 0.0672536
preserved ~ CumErr	1903.209	2.507702	0.2854036	0.2220342	0.3950320	2.896421	- 2.067928	NA
preserved ~ CumPres	2795.127	894.425527	0.0000000	0.0000000	0.0004137	2.205245	NA	- 0.0228220

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      3.3643      -1.9354      -0.1212
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4159 Residual
## Null Deviance:      2792
## Residual Deviance: 1884 AIC: 1890
## log likelihood: -942.1575
## Nagelkerke R2: 0.4009303
## % pres/err predicted correctly: -488.5384
## % of predictable range [ (model-null)/(1-null) ]: 0.3729105
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.896      -2.068
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4160 Residual
## Null Deviance:      2792
## Residual Deviance: 1899 AIC: 1903
## log likelihood: -949.6046
## Nagelkerke R2: 0.395032
## % pres/err predicted correctly: -489.9273
## % of predictable range [ (model-null)/(1-null) ]: 0.3711313
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.7424      -0.3579
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4160 Residual
## Null Deviance:      2792
## Residual Deviance: 2556 AIC: 2560
## log likelihood: -1278.229
## Nagelkerke R2: 0.1125705
## % pres/err predicted correctly: -732.9789
## % of predictable range [ (model-null)/(1-null) ]: 0.05978689
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr	1890.315	0.00000	1.0000000	0.9984175	0.4009303	3.364305	-	-
+ pos							1.935390	0.1212490
preserved ~ CumErr	1903.209	12.89427	0.0015851	0.0015825	0.3950320	2.896421	-	NA
							2.067928	
preserved ~ pos	2560.458	670.14317	0.0000000	0.0000000	0.1125705	3.742386	NA	-
								0.3578735

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + I(pos^2) + pos	1863.270	0.0000001	0.0000000	0.0000000	0.4123715	693207	-	0.0777639	-	NA	NA
							1.938494		0.8366701		
preserved ~ CumErr + pos	1890.316	0.0000001	0.0000000	0.9984175	0.4009303	3364305	-	NA	-	NA	NA
							1.935390		0.1212490		
preserved ~ CumErr + stimlen	1899.400	0.0000001	0.0000000	0.8704392	0.3973352	619925	-	NA	NA	-	NA
							2.027987			0.0940391	
preserved ~ CumErr + CumPres	1900.700	0.0000001	0.0000000	0.7779653	0.3968193	8080188	-	NA	NA	NA	-
							2.067310				0.0672536
preserved ~ CumErr	1903.209	9.937986	0.0000000	0.0000000	0.3950320	896421	-	NA	NA	NA	NA
							2.067928				
preserved ~ CumErr	1903.209	8.8096960	0.1488453	0.1295608	0.3950320	896421	-	NA	NA	NA	NA
							2.067928				
preserved ~ CumErr	1903.209	2.5077020	0.2854036	0.2220342	0.3950320	896421	-	NA	NA	NA	NA
							2.067928				
preserved ~ CumErr	1903.209	2.894270	0.0015851	0.0015825	0.3950320	896421	-	NA	NA	NA	NA
							2.067928				
preserved ~ I(pos^2) + pos	2547.356	84.083389	0.0000000	0.0000000	0.1195748	629673	NA	0.0428527	-	NA	NA
									0.7869789		
preserved ~ pos	2560.458	70.143170	0.0000000	0.0000000	0.1125705	742386	NA	NA	-	NA	NA
									0.3578735		
preserved ~ stimlen	2720.498	21.092893	0.0000000	0.0000000	0.0367724	322833	NA	NA	NA	-	NA
										0.2739542	
preserved ~ CumPres	2795.128	94.425527	0.0000000	0.0000000	0.0004137	205245	NA	NA	NA	NA	-
											0.0228220

```
# explore influence of frequency and length

if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)          pos      stimlen
##      5.30651      -1.93382      0.08227      -0.85008      -0.08384
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4157 Residual
## Null Deviance:      2792
## Residual Deviance: 1852 AIC: 1862
## log likelihood: -925.8585
## Nagelkerke R2: 0.4137662
## % pres/err predicted correctly: -484.0068
## % of predictable range [ (model-null)/(1-null) ]: 0.3787155
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)          pos      stimlen      log_freq
##      5.21824      -1.93336      0.08193      -0.84733      -0.07260      0.03682
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4156 Residual
## Null Deviance:      2792
## Residual Deviance: 1851 AIC: 1863
## log likelihood: -925.321
## Nagelkerke R2: 0.4141878
## % pres/err predicted correctly: -483.8892
## % of predictable range [ (model-null)/(1-null) ]: 0.3788661
## *****
## model index: 3

```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)      pos      log_freq
##      4.68347      -1.93699      0.07811      -0.83501      0.05032
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4157 Residual
## Null Deviance:      2792
## Residual Deviance: 1853 AIC: 1863
## log likelihood: -926.5719
## Nagelkerke R2: 0.4132065
## % pres/err predicted correctly: -484.1699
## % of predictable range [ (model-null)/(1-null) ]: 0.3785064
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)      pos
##      4.69321      -1.93849      0.07776      -0.83667
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4158 Residual
## Null Deviance:      2792
## Residual Deviance: 1855 AIC: 1863
## log likelihood: -927.6356
## Nagelkerke R2: 0.4123715
## % pres/err predicted correctly: -484.3936
## % of predictable range [ (model-null)/(1-null) ]: 0.37822
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.145
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4161 Residual
## Null Deviance:      2792
## Residual Deviance: 2792 AIC: 2794
## log likelihood: -1395.984
## Nagelkerke R2: 2.271729e-16
## % pres/err predicted correctly: -779.6516
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]
```

```

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                            CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	log_freq	stimlen
preserved ~ CumErr + I(pos^2) + pos + stimlen	1861.717	0.000000	1.000000	0.387690	0.511376	5206505	-	0.0822749	-	NA	-
							1.933817	0.8500830	0.0838381		
preserved ~ CumErr + I(pos^2) + pos + stimlen + log_freq	1862.642	0.925024	0.629699	0.244129	0.141852	18237	-	0.0819289	-	0.0368193	-
							1.933362	0.8473276	0.0725950		
preserved ~ CumErr + I(pos^2) + pos + log_freq	1863.144	1.426821	0.489970	0.189957	0.113204	583467	-	0.0781086	-	0.0503210	NA
							1.936988	0.8350141			
preserved ~ CumErr + I(pos^2) + pos	1863.271	1.554359	0.459700	0.178220	0.112374	593207	-	0.0777639	-	NA	NA
							1.938494	0.8366701			
preserved ~ 1	2793.968	12.251637	0.000000	0.000000	0.000000	000000	NA	NA	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

```

```

## Single term deletions
##
## Model:
## preserved ~ CumErr + I(pos^2) + pos + stimlen
##      Df Deviance   AIC
## CumErr    1   2534.4 2542.4
## pos       1   1891.0 1899.0
## I(pos^2)   1   1883.4 1891.4
## stimlen    1   1855.3 1863.3
## <none>      1   1851.7 1861.7

```

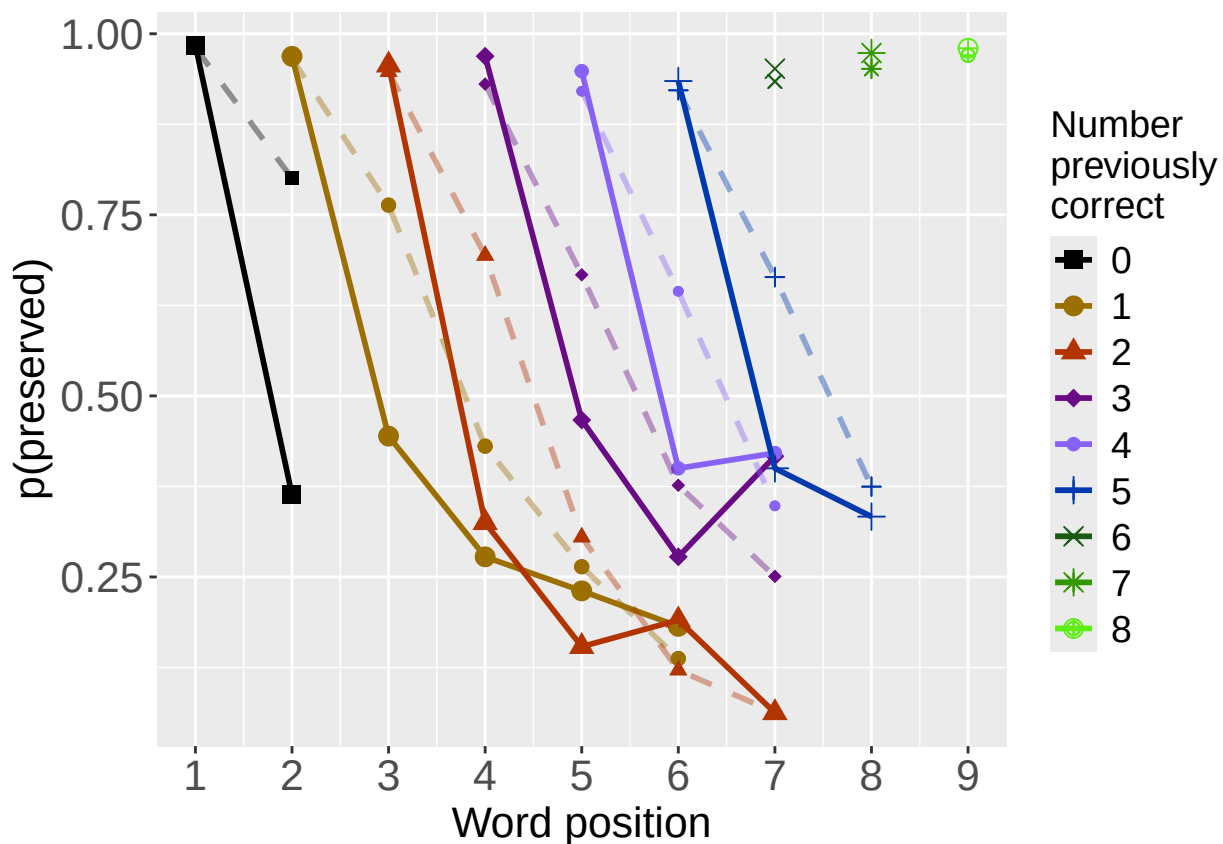
```
#####
# Single deletions from best model
#####

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

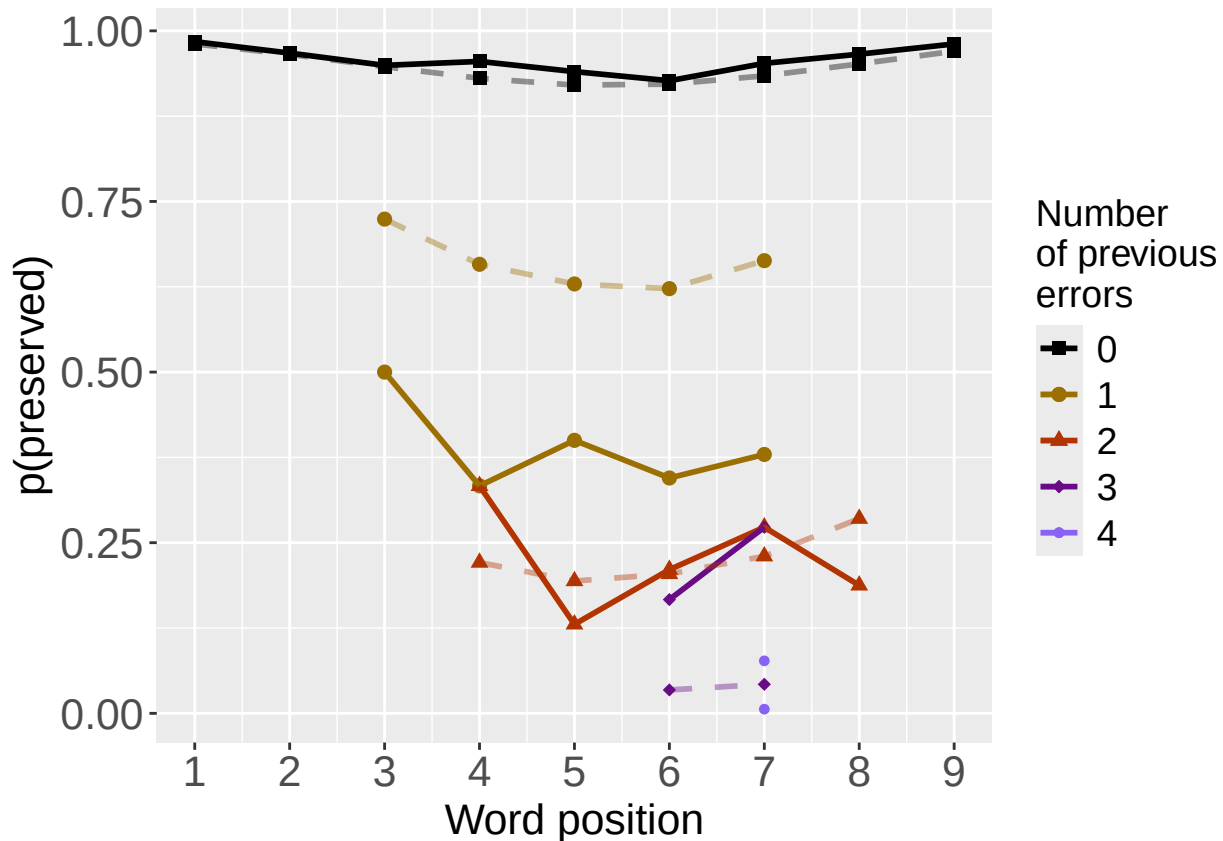
PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
                 RunLabel,"_",CurTask,
                 "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.896      -2.068
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4160 Residual
## Null Deviance:      2792
## Residual Deviance: 1899 AIC: 1903
## log likelihood: -949.6046
## Nagelkerke R2: 0.395032
## % pres/err predicted correctly: -489.9273
## % of predictable range [ (model-null)/(1-null) ]: 0.3711313

```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      3.3643      -1.9354      -0.1212
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4159 Residual
## Null Deviance:      2792
## Residual Deviance: 1884 AIC: 1890
## log likelihood: -942.1575
## Nagelkerke R2: 0.4009303
## % pres/err predicted correctly: -488.5384
## % of predictable range [ (model-null)/(1-null) ]: 0.3729105
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos      I(pos^2)
##      4.69321      -1.93849      -0.83667      0.07776
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4158 Residual
## Null Deviance:      2792
## Residual Deviance: 1855 AIC: 1863
## log likelihood: -927.6356
## Nagelkerke R2: 0.4123715
## % pres/err predicted correctly: -484.3936
## % of predictable range [ (model-null)/(1-null) ]: 0.37822
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos      I(pos^2)      stimlen
##      5.30651      -1.93382      -0.85008      0.08227      -0.08384
##
## Degrees of Freedom: 4161 Total (i.e. Null); 4157 Residual
## Null Deviance:      2792
## Residual Deviance: 1852 AIC: 1862
## log likelihood: -925.8585
## Nagelkerke R2: 0.4137662
## % pres/err predicted correctly: -484.0068
## % of predictable range [ (model-null)/(1-null) ]: 0.3787155
## *****
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

```
## summarise() has grouped output by 'CumPres'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
```

```
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes  
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.  
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point`)
```

```
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes  
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.  
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point`)
```

```
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes  
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.  
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point`)  
## Removed 9 rows containing missing values or values outside the scale range (`geom_point`).
```

```
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes  
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.  
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point`)  
## Removed 3 rows containing missing values or values outside the scale range (`geom_point`).
```

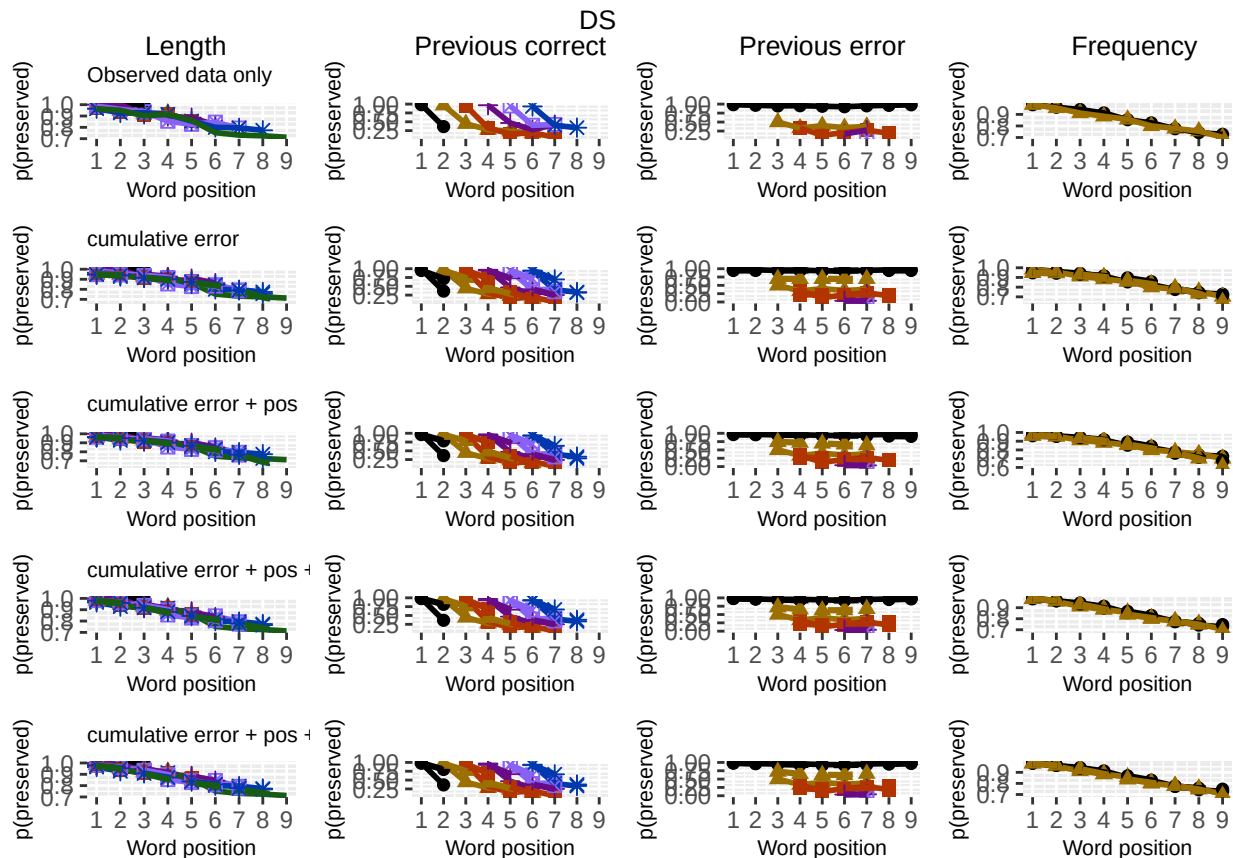
```
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes  
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.  
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point`)  
## Removed 9 rows containing missing values or values outside the scale range (`geom_point`).
```

```
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes  
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.  
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point`)  
## Removed 3 rows containing missing values or values outside the scale range (`geom_point`).
```

```
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes  
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.  
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point`)  
## Removed 9 rows containing missing values or values outside the scale range (`geom_point`).
```

```
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
```

```
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`)
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```



```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
CurTask,"_dominance_analysis_table.csv"),
row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	I(pos ²)	pos	stimlen
McFadden	0.2678104	0.0266395	0.0344863	0.0078339

	CumErr	I(pos ²)	pos	stimlen
SquaredCorrelation	0.1583641	0.0169378	0.0217927	0.0051182
Nagelkerke	0.1583641	0.0169378	0.0217927	0.0051182
Estrella	0.1930158	0.0184545	0.0239684	0.0053407

	deviance	deviance_explained
CumErr + pos + I(pos ²) + stimlen	1851.717	940.2516
CumErr + pos + I(pos ²)	1855.271	936.6973
CumErr + pos	1884.315	907.6536
CumErr	1899.209	892.7593
null	2791.969	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained
## CumErr + pos + I(pos^2) + stimlen CumErr + pos + I(pos^2) + stimlen 1851.717      940.2516      33.67701
## CumErr + pos + I(pos^2)          CumErr + pos + I(pos^2) 1855.271      936.6973      33.54971
## CumErr + pos                    CumErr + pos 1884.315      907.6536      32.50945
## CumErr                          CumErr 1899.209      892.7593      31.97598
## null                            null 2791.969      0.0000      0.00000
##                                percent_of_explained_deviance increment_in_explained
## CumErr + pos + I(pos^2) + stimlen 100.00000      0.3780222
## CumErr + pos + I(pos^2)          99.62198      3.0889296
## CumErr + pos                    96.53305      1.5840732
## CumErr                          94.94898      94.9489750
## null                            NA      0.0000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + pos + I(pos ²) + stimlen	33.67701	100.00000	0.3780222
CumErr + pos + I(pos ²)	33.54971	99.62198	3.0889296
CumErr + pos	32.50945	96.53305	1.5840732
CumErr	31.97598	94.94898	94.9489750
null	0.00000	NA	0.0000000

```
"_nagelkerke_contributions_table.csv"),row.names = TRUE)
```

```
NagPercents
```

```
##      Nagelkerke
## CumErr  0.78315566
## I(pos^2) 0.08376248
## pos     0.10777106
## stimlen 0.02531080
```

```
sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

```

63

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_sse_results_table.csv"),row.names = TRUE)
sse_table

```

	model	p_accounted_for	model_deviance	diff_CumErr	diff_CumErr+pos	diff_CumErr+pos+I(pos^2)
## 1	preserved ~ CumErr	0.9444331	1899.209	0.000000000	-0.004704321	-0.015229685
## 2	preserved ~ CumErr+pos	0.9491374	1884.315	0.004704321	0.000000000	-0.010525364
## 3	preserved ~ CumErr+pos+I(pos^2)	0.9596628	1855.271	0.015229685	0.010525364	0.000000000

model	p_accounted_for	model_deviance
preserved ~ CumErr	0.9444331	1899.209
preserved ~ CumErr+pos	0.9491374	1884.315
preserved ~ CumErr+pos+I(pos^2)	0.9596628	1855.271
preserved ~ CumErr+pos+I(pos^2)+stimlen	0.9612180	1851.717

model	diff_CumErr	diff_CumErr+pos	diff_CumErr+pos+I(pos^2)	diff_CumErr+pos+I(pos^2)+stimlen
preserved ~ CumErr	0.0000000	-0.0047043	-0.0152297	-0.0167849
preserved ~ CumErr+pos	0.0047043	0.0000000	-0.0105254	-0.0120806
preserved ~ CumErr+pos+I(pos^2)	0.0152297	0.0105254	0.0000000	-0.0015552
preserved ~ CumErr+pos+I(pos^2)+stimlen	0.0167849	0.0120806	0.0015552	0.0000000

```
## 4 preserved ~ CumErr+pos+I(pos^2)+stimlen      0.9612180      1851.717 0.016784889      0.012080567      0.001555203
##   diff_CumErr+pos+I(pos^2)+stimlen
## 1      -0.016784889
## 2      -0.012080567
## 3      -0.001555203
## 4      0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                   CurTask,"_results_report_df.csv"),row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```